

AI 데이터 분석 · 13일차

파일 입출력과 예외처리

읽고 쓰고, 그러다 나는 오류를 다룬다

read · write · csv.reader · csv.writer · 트레이스백 · try-except · 구체적인 예외

교안 08_02(파일 입출력) 나머지 + 09_01(예외처리 기초) 전 범위

데이터 처리 한 사이클이 완성되는 날

2026-08-06 AI 데이터 분석 교육 정리 (13일차)

학습자 Minho · 국립금오공과대학교 컴퓨터공학부

이 책을 읽는 법

어제는 파일을 찾아 여는 것까지 배웠습니다. 오늘 오전에 그 파일을 읽고 쓰는 법을 배워 데이터 처리 한 사이클을 완성했고, 오후에는 그 과정에서 나는 오류를 다루는 법을 배웠습니다. 순서가 우연이 아닙니다 — 강사님 표현대로 "파일을 다루다 보면 에러가 먼저 발생하기" 때문입니다.

파일 read·write·모드 **CSV** reader·writer·DictReader **예외** try·except **개념** 트레이스백 읽기 **키워드**

ValueError·FileNotFoundError

연결

오늘로 "찾기 → 열기 → 읽기 → 처리 → 저장" 다섯 단계가 전부 손에 들어옵니다. 어제 배운 os가 찾기, open이 열기였고, 오늘 csv.reader가 읽기, float() 변환이 처리, csv.writer가 저장입니다. 그리고 오후의 try-except가 이 사이클이 중간에 끊기지 않게 하는 안전장치입니다.

주의

오늘 교안은 09_02(견고한 예외처리)까지 올라와 있지만 진도는 09_01까지입니다. else·finally·bare except·as e·raise는 수업 녹취 두 파일을 통틀어 한 번도 언급되지 않았고 수업 코드에도 없습니다. 강사님이 09_01 실습 3에서 마무리하셨으므로 이 책도 거기까지만 담고, 09_02는 부록 C에 예고로 정리했습니다.

목차

	장 / STEP	무엇을 배우나
CHAPTER 0	오늘의 지도 — 사이클을 완성한다	읽기·쓰기·오류 대응
STEP 1	파일 읽기 — read 3종	끝의 s 한 글자가 결과를 바꾼다
STEP 2	파일 쓰기 — w·a 모드	단기의 의무와 with
STEP 3	csv.reader	표를 행 단위 리스트로
STEP 4	csv.writer	리스트를 행으로 저장·newline
STEP 5	헤더 다루기	next()·DictReader
STEP 6	오류와 예외의 이해	3종류·트레이스백 읽는 순서
STEP 7	try-except 기본 구조	일어나면 그때 대응한다
STEP 8	구체적 예외 처리	ValueError·FileNotFoundError 외
STEP 9	여러 예외 동시 처리	순서·구분·묶기
STEP 10	내 코드 점검 — 수업 코드 2개	파일명 오타·bare except
STEP 11	실습 점검 — 기본 5·선택 2	원본 CSV 오염 사고 포함
부록 A·B·C	치트시트·오류 사전·예고	한 장 요약·함정 모음

CHAPTER 0

오늘의 지도 — 사이클을 완성한다

찾기 → 열기 → 읽기 → 처리 → 저장

어제 교안 마지막 장이 오늘의 지도를 미리 보여줬습니다 — "with open으로 안전하게 열고, 읽기는 reader로 리스트를 받아 형변환, 쓰기는 writer로 리스트를 행으로 저장한다." 오늘 그 나머지를 채웁니다.

단계	도구	언제 배웠나
① 찾기	os.getcwd · listdir · path.join · exists	12일차
② 열기	open · with · 모드 · 인코딩	12일차
③ 읽기	read · readline · readlines · csv.reader	오늘 오전
④ 처리	float 변환 · 조건 필터	오늘 오전
⑤ 저장	csv.writer · newline=""	오늘 오전
⑥ 안전장치	try · except	오늘 오후

왜 파일 다음이 예외처리인가

교안이 든 장면이 정확합니다. **센서 파일 50줄을 처리하는데 10번째 줄에 불량 데이터가 있으면**, 프로그램은 그 줄에서 멈추고 **나머지 40줄은 처리되지 못합니다**. 50줄 중 49줄이 멀쩡해도 결과는 0줄인 셈입니다.

설비 적용

현장 데이터는 **깨끗하지 않습니다**. 센서가 끊겨 빈 칸이 생기고, 글자가 섞이고, 열 개수가 안 맞는 줄이 들어옵니다. 그런 줄 하나 때문에 하루치 분석이 멈추면 안 되죠. 오후에 배운 try-except가 "그 줄만 건너뛰고 나머지는 계속 처리하라"를 코드로 표현하는 도구입니다.

오늘의 흐름

앞의 세 칸이 오전, 마지막 칸이 오후입니다 — **읽어서 숫자로 바꾸고, 조건에 맞는 것만 새 파일로 저장하되, 중간에 이상한 값이 나와도 멈추지 않는다**.



STEP 1

파일 읽기 — read 3종

끝의 s 한 글자가 결과를 바꾼다

어제 open으로 **연결 통로**를 만드는 데까지 배웠습니다. 이제 그 통로로 내용을 꺼냅니다. 방법이 세 가지인데, **꺼내는 범위와 돌려주는 자료형이 다릅니다**.

방법	읽는 범위	돌려주는 것	언제
<code>f.read()</code>	전체를 한 덩어리로	문자열 (str)	내용을 통째로 볼 때
<code>f.readline()</code>	한 줄만	문자열 (str)	첫 줄(헤더)만 볼 때
<code>f.readlines()</code>	전체를 줄 단위로	리스트 (list)	줄마다 처리할 때

메서드 `.readlines()` 모든 줄을 리스트로

정의 파일 전체를 **줄 단위로 잘라 리스트**로 돌려줍니다. 각 요소 끝에는 줄바꿈 문자 `\n`이 붙어 있습니다.

형태 `f.readlines()`

```
with open("Data/08_press.csv", "r", encoding="utf-8") as f:
    lines = f.readlines()
```

```
print(type(lines).__name__, len(lines))
print(lines[0])
print(lines[1])
```

▶ list 13

▶ 설비ID, 시각, 진동X, 진동Y, 전류, 상태\n

▶ PRESS-01,2022-07-12 00:00:00.019,0.117,-0.1764,192.3387,0\n

왜 쓰나 줄 단위 **센서 로그**에 가장 잘 맞습니다. 리스트가 되면 어제까지 배운 인덱싱·슬라이싱·반복문을 그대로 쓸 수 있으니까요. `lines[0]`이 헤더, `lines[1:]`이 데이터가 됩니다.

응용 각 줄 끝의 `\n`이 거슬리면 `.strip()`으로 떼어냅니다. 그리고 파일이 아주 클 때는 `readlines()`가 **전체를 메모리에 올리므로**, 한 줄씩 도는 `for line in f:` 쪽이 가볍습니다.

```
# ① readlines + 슬라이싱
with open("Data/08_press.csv", "r", encoding="utf-8") as f:
    lines = f.readlines()

header = lines[0].strip()
print("헤더:", header)
print("데이터 행 수:", len(lines) - 1)

# ② 파일을 직접 반복 — 큰 파일에 유리
with open("Data/08_press.csv", "r", encoding="utf-8") as f:
    for line in f:
        print(line.strip())
        break
```

- ▶ 헤더: 설비ID, 시각, 진동X, 진동Y, 전류, 상태
- ▶ 데이터 행 수: 12
- ▶ 설비ID, 시각, 진동X, 진동Y, 전류, 상태

자주 하는 실수

`readlines(5)`는 "5줄을 읽어라"가 아닙니다. **괄호 안 숫자는 줄 수가 아니라** 대략 몇 바이트까지 읽을지에 대한 힌트(hint)**입니다**. `readlines(5)`를 하면 5바이트를 채우는 첫 줄 하나만 나옵니다. 실습 1에서 실제로 이 오해가 나왔는데, STEP 11에서 자세히 다룹니다.

메서드 `.read()` / `.readline()` 통째로 · 한 줄만

정의 `read()`는 파일 전체를 **하나의 긴 문자열**로, `readline()`은 **한 줄만** 문자열로 돌려줍니다.

```
with open("Data/sample.txt", "r", encoding="utf-8") as f:
    content = f.read()
    print(type(content).__name__)
    print(content)

with open("Data/sample.txt", "r", encoding="utf-8") as f:
    first = f.readline()
    print(repr(first))
```

- ▶ str
- ▶ 123
- ▶ 456
- ▶ 1
- ▶ '123\n'

왜 쓰나 `read()`는 파일이 작고 **내용을 통째로 보고 싶을 때** 편합니다. `readline()`은 **헤더 한 줄만** 확인하고 싶을 때 쓰죠.

응용 `read()`가 돌려준 문자열은 5일차에 배운 문자열 메서드를 그대로 쓸 수 있습니다. `split("\n")`으로 줄을 나누면 `readlines()`와 비슷해지지만, 줄바꿈이 제거된다는 차이가 있습니다.

```
with open("Data/sample.txt", "r", encoding="utf-8") as f:
    content = f.read()
```

```
print("글자 수:", len(content))
print("줄 수 :", len(content.split("\n")))
print("줄 목록:", content.split("\n"))
```

```
▶글자 수: 12
▶줄 수 : 4
▶줄 목록: ['123', '456', '1', '']
```

개념

`readline()`은 **호출할 때마다 다음 줄**을 돌려줍니다. 파일 객체가 "지금 어디까지 읽었는지"를 기억하고 있기 때문입니다. 그래서 두 번 부르면 1줄·2줄이 차례로 나옵니다 — STEP 5의 `next(reader)`가 같은 원리로 동작합니다.

STEP 2

파일 쓰기 — w · a 모드

닫기의 의무와 with

메서드 `.write()` 문자열을 파일에 쓰기

정의 문자열 하나를 파일에 씁니다. **줄바꿈이 자동으로 붙지 않으므로** 줄을 나누려면 `\n`을 직접 넣어야 합니다.

형태 `f.write("문자열")`

```
f = open("hello.txt", "w", encoding="utf-8")

f.write("안녕하세요\n")          # \n 을 직접 넣는다
f.write("\t반갑습니다\n")       # \t 는 탭 들여쓰기

f.close()
```

▶ (hello.txt 내용)

▶ 안녕하세요

▶ 반갑습니다

왜 쓰나 `print()`는 자동으로 줄을 바꾸지만 `write()`는 바꾸지 않습니다. 이 차이를 모르면 모든 내용이 한 줄에 붙어 나옵니다.

응용 여러 줄을 한 번에 쓰려면 `writelines()`나 `"\n".join()`을 씁니다. 다만 `writelines()`도 **줄바꿈을 넣어주지 않으므로** 각 요소에 `\n`이 있어야 합니다.

```
lines = ["첫째 줄", "둘째 줄", "셋째 줄"]

# ① join으로 이어 붙이기 (권장)
with open("a.txt", "w", encoding="utf-8") as f:
    f.write("\n".join(lines))

# ② writelines - 각 요소에 \n 이 있어야 줄이 나뉜다
with open("b.txt", "w", encoding="utf-8") as f:
    f.writelines([l + "\n" for l in lines])
```

자주 하는 실수

writelines(["1","2","3"])처럼 \n 없이 넘기면 123이 한 줄로 붙습니다. 실습 2에서 실제로 나온 부분이라 STEP 11에서 다시 봅니다.

문법 w 와 a 의 차이 지우고 쓸 것인가, 이어 쓸 것인가

정의 w는 기존 내용을 전부 지우고 새로 씁니다. a는 기존 뒤에 이어 붙입니다.

```
# w 모드 — 실행할 때마다 앞 내용이 사라진다
with open("log.txt", "w", encoding="utf-8") as f:
    f.write("점검 1회\n")

# a 모드 — 뒤에 쌓인다
with open("log.txt", "a", encoding="utf-8") as f:
    f.write("점검 2회\n")

with open("log.txt", "r", encoding="utf-8") as f:
    print(f.read())
```

▶ 점검 1회

▶ 점검 2회

왜 쓰나 교안이 경고 표시까지 붙인 이유가 있습니다 — w와 a를 헷갈리면 데이터가 날아갑니다. 로그처럼 누적되어야 하는 데이터를 w로 열면 매번 앞 기록이 사라집니다.

응용 반대 방향의 사고도 있습니다. 원본 데이터 파일을 a로 열면 원본이 오염됩니다. 결과는 반드시 새 파일에 쓰는 것이 원칙입니다. 실습 5에서 이 사고가 실제로 났는데, STEP 11에서 자세히 다룹니다.

```
# x 원본에 이어쓰기 — 실행할 때마다 원본이 더러워진다
with open("Data/08_press.csv", "a", encoding="utf-8") as f:
    ...

# ✓ 결과는 새 파일로
with open("Data/result.csv", "w", encoding="utf-8", newline="") as f:
    ...
```

주의

w 모드는 **파일이 없으면 새로 만들어 줍니다**. 그래서 결과 파일을 미리 만들어 둘 필요가 없습니다. r 모드만 없는 파일에서 FileNotFoundError가 납니다.

키워드 close() 와 with 열었으면 닫는다

정의 파일을 닫아야 하는 이유는 ① **자원 반납**과 ② **저장 보장** 두 가지입니다. with는 이를 자동으로 처리합니다.

```
# ① 직접 닫기 — 깜빡하기 쉽다
f = open("Data/sample.txt", "r", encoding="utf-8")
lines = f.readlines()
f.close()

# ② with — 블록을 벗어나면 자동으로 닫힌다
with open("Data/sample.txt", "r", encoding="utf-8") as f:
    lines = f.readlines()

print(lines)

▶ ['123\n', '456\n', '1\n', '123']
```

왜 쓰나 쓰기에서 닫지 않으면 **임시 공간의 내용이 실제 파일에 기록되지 못하고 사라집니다**. 읽기에서도 안 닫으면 열린 파일이 쌓여 나중에 새 파일을 못 열게 됩니다.

응용 with가 더 안전한 결정적 이유는 **오류가 나도 닫아 주기** 때문입니다. close()를 직접 쓰면 중간에서 예외가 터졌을 때 그 줄까지 도달하지 못합니다 — 오늘 오후에 배운 예외 개념과 정확히 이어집니다.

```
# x 중간에 오류가 나면 close()에 도달 못 함
f = open("Data/sample.txt", "r", encoding="utf-8")
value = int("abc")          # ValueError! 여기서 중단
f.close()                   # 실행되지 않는다 → 파일이 열린 채 남음

# ✓ with — 오류가 나도 블록을 벗어나며 닫힌다
with open("Data/sample.txt", "r", encoding="utf-8") as f:
    value = int("abc")       # 오류가 나도 파일은 닫힌다
```

개념

그래서 실전 코드는 거의 항상 with open입니다. f = open(...) 형태는 **원리를 이해하기 위한 것**이고, 실무에서 직접 close()를 부르는 경우는 드뭅니다.

STEP 3

csv.reader

표를 행 단위 리스트로

CSV를 `readlines()`로 읽으면 각 줄이 **긴 문자열 하나**입니다. 쉼표로 자르려면 `split(",")`을 해야 하는데, 값 안에 쉼표가 들어 있거나 따옴표로 감싼 경우까지 처리하려면 금세 복잡해집니다. **그 일을 대신해 주는 전문가가 `csv` 모듈입니다.**

split vs csv.reader

직접 자르면

```
line = 'PRESS-01,2022-07-12 00:00:00.019,0.117,-0.1764,192.3387,0'
print(line.split(","))
```

`csv.reader`에 맡기면 — 따옴표·특수 케이스까지 알아서 처리

```
▶ ['PRESS-01', '2022-07-12 00:00:00.019', '0.117', '-0.1764', '192.3387', '0']
```

csv `csv.reader()` 행을 리스트로 꺼내기

정의 `with open`으로 연 파일을 넘기면, **각 행을 리스트로** 하나씩 돌려줍니다. 모든 값은 **문자열**입니다.

형태 `import csv reader = csv.reader(f) for row in reader:`

```
import csv
```

```
with open("Data/08_press.csv", "r", encoding="utf-8") as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)
```

```
▶ ['설비ID', '시각', '진동X', '진동Y', '전류', '상태']
▶ ['PRESS-01', '2022-07-12 00:00:00.019', '0.117', '-0.1764', '192.3387', '0']
▶ ['PRESS-02', '2022-07-12 00:08:27.624', '0.242', '0.1', '-51.347', '0']
▶ ...
```

왜 쓰나 쉼표로 자르는 일을 **reader가 알아서** 합니다. 그리고 결과가 리스트이므로 6일차에 배운 인덱싱을 그대로 쓸 수 있습니다 — `row[0]`이 설비ID, `row[4]`가 전류입니다.

응용 모든 값이 **문자열**이라는 점이 핵심입니다. 계산하려면 반드시 `float()`이나 `int()`로 바꿔야 합니다. 이걸 빠뜨리면 문자열끼리 비교해 엉뚱한 결과가 나옵니다.

```
import csv

with open("Data/08_press.csv", "r", encoding="utf-8") as f:
    reader = csv.reader(f)
    next(reader)  # 헤더 건너뛰기 (STEP 5)
    for row in reader:
        name = row[0]
        current = float(row[4])  # ← 문자열 → 실수
        if abs(current) > 100:
            print(f"{name}: 전류 {current}")
```

▶ PRESS-01: 전류 192.3387

▶ PRESS-03: 전류 109.3448

자주 하는 실수

형변환을 빠뜨리면 **오류 없이 결과만 틀립니다**. "192.3387" > "100"은 문자열 비교라 사전 순서로 판정됩니다 — "1"과 "1"을 비교하고, "9"와 "0"을 비교해 True가 나오죠. 교안의 "흔한 오류 4종" 중 **"계산 오류 — 문자열 상태"**가 이것입니다.

문자열 비교의 함정

왜 float 변환이 필수인가

x 형변환을 빠뜨리면

```
print("192.3387" > "100")  # True   (우연히 맞음)
print("21.4577" > "100")  # True   (틀림! 21은 100보다 작다)
print("9" > "100")        # True   (틀림)
```

✓ 숫자로 바꾸고 비교

```
print(float("21.4577") > 100)  # False
```

▶ True
▶ True
▶ True
▶ False

자주 하는 실수

문자열 비교는 **첫 글자부터 사전 순서로** 비교합니다. "21.4577"과 "100"은 첫 글자 "2"와 "1"을 비교해 "2" > "1"이므로 True가 나옵니다. 숫자로 21 < 100인데 정반대죠. **오류가 나지 않아 더 위험합니다**. csv에서 읽은 값으로 크기를 비교할 때는 반드시 float()을 거치세요.

STEP 4

csv.writer

리스트를 행으로 저장

CSV csv.writer() reader의 정확한 반대

정의 리스트를 **한 행으로** 파일에 씁니다. writerow(리스트)가 한 행, writerows(리스트의 리스트)가 여러 행입니다.

형태 writer = csv.writer(f) writer.writerow([값1, 값2, ...])

```
import csv

with open("Data/result.csv", "w", encoding="utf-8", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["시각", "설비"])
    writer.writerow(["09:00", "PUMP-01"])

with open("Data/result.csv", "r", encoding="utf-8") as f:
    print(f.read())
```

▶ 시각, 설비
▶ 09:00, PUMP-01

왜 쓰나 심표를 직접 붙이거나 줄바꿈을 넣을 필요가 없습니다. **리스트를 주면 알아서 한 행으로** 만들어 줍니다. reader의 정확한 반대 방향이죠.

응용 여러 행을 한 번에 쓸 때는 writerows()를 씁니다. 그리고 **모든 행의 열 개수를 맞춰야** 표가 어긋나지 않습니다.

```
import csv

header = ["설비ID", "전류"]
rows = [
    ["PRESS-01", 192.3387],
    ["PRESS-03", 109.3448],
]

with open("Data/over.csv", "w", encoding="utf-8", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(header)      # 헤더 한 행
    writer.writerows(rows)      # 데이터 여러 행
```

▶ (over.csv)
▶ 설비ID, 전류
▶ PRESS-01, 192.3387
▶ PRESS-03, 109.3448

자주 하는 실수

쓰기 모드는 `w`, 그리고 `newline=""`이 필수입니다. 이유는 바로 아래에서 다룹니다.

newline="" 이 필요한 이유

교안이 "CSV 쓰기에는 항상 `newline=''` — 패턴으로 기억"이라고 못 박은 부분입니다. 이유를 알면 잊지 않습니다.

	일어나는 일	결과
<code>csv.writer</code>	행 끝에 <code>\r\n</code> 을 붙여 쓴다	—
텍스트 모드 (윈도우)	<code>\n</code> 을 <code>\r\n</code> 으로 다시 바꾼다	<code>\r\r\n</code> 이 된다
<code>newline=""</code>	그 자동 변환을 끈다	<code>\r\n</code> 그대로

주의

맥·리눅스에서는 이 차이가 안 보입니다. 자동 변환이 없기 때문에 `newline=""`이 있든 없든 결과가 같거든요. 그래서 맥에서 짠 코드를 윈도우에서 돌렸을 때 행 사이에 빈 줄이 하나씩 끼는 현상으로 처음 발견하게 됩니다. 교안의 "흔한 오류 4종" 중 "빈 줄 끼임 — `newline` 누락"이 이것입니다. 습관처럼 붙이세요.

reader와 writer 대칭

	읽기	쓰기
모드	<code>"r"</code>	<code>"w"</code> (또는 <code>"a"</code>)
추가 옵션	없음	<code>newline=""</code>
객체 만들기	<code>reader = csv.reader(f)</code>	<code>writer = csv.writer(f)</code>
한 행	<code>for row in reader:</code>	<code>writer.writerow(...)</code>
여러 행	반복문이 알아서	<code>writer.writerows([[...], [...]])</code>
값의 자료형	전부 문자열 → float 필요	숫자를 줘도 알아서 문자로

STEP 5

헤더 다루기

next() 와 DictReader

CSV의 첫 줄은 대개 열 제목(헤더)입니다. `csv.reader`는 이 줄도 그냥 데이터처럼 돌려주므로, 계산할 때 `float("진동X")`에서 터집니다. 교안의 "흔한 오류 4종" 중 "첫 줄 오류 — 헤더 포함"이죠. 해결법이 두 가지입니다.

내장함수 `next(reader)` 첫 줄을 챙기며 건너뛰기

정의 `reader`에서 다음 한 행을 꺼냅니다. 반복문 앞에서 한 번 부르면 헤더를 변수에 담으면서 건너뛸 수 있습니다.

형태 `header = next(reader)`

```
import csv

with open("Data/08_press.csv", "r", encoding="utf-8") as f:
    reader = csv.reader(f)

    header = next(reader)      # 첫 줄을 꺼내 header에 담는다
    print(header)

    for row in reader:         # 두 번째 줄부터 반복
        print(row[0])
```

- ▶ ['설비ID', '시각', '진동X', '진동Y', '전류', '상태']
- ▶ PRESS-01
- ▶ PRESS-02
- ▶ ...

왜 쓰나 헤더를 버리지 않고 챙기면서 건너뛴다는 점이 좋습니다. 나중에 결과 CSV를 저장할 때 그 헤더를 그대로 다시 쓸 수 있으니까요.

응용 `next()`가 동작하는 원리는 STEP 1의 `readline()`과 같습니다 — 파일 객체가 "지금 어디까지 읽었는지"를 기억하기 때문에, 한 번 꺼내면 그 다음부터 반복이 시작됩니다.

```
import csv

with open("Data/08_press.csv", "r", encoding="utf-8") as f:
    reader = csv.reader(f)
    header = next(reader)

    rows = []
    for row in reader:
        if float(row[4]) > 100:
            rows.append(row)

# 땡겨둔 header를 결과 파일에 그대로 쓴다
with open("Data/over.csv", "w", encoding="utf-8", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(header)
    writer.writerows(rows)
```

개념

enumerate로 if idx == 0: continue 하는 방법도 됩니다. 다만 그 방식은 **헤더를 땡기지 못하고 버리게** 되므로, 결과 파일에 헤더를 다시 쓰려면 손으로 적어야 합니다.

csv csv.DictReader() 번호 대신 이름으로

정의 첫 줄을 자동으로 헤더로 인식하고, 각 행을 **헤더 이름을 키로 하는 딕셔너리**로 돌려줍니다.

형태 reader = csv.DictReader(f) row["열이름"]

```
import csv

with open("Data/08_press.csv", "r", encoding="utf-8") as f:
    reader = csv.DictReader(f)

    for row in reader:
        print(row["설비ID"], row.get("시각"))
```

```
▶ PRESS-01 2022-07-12 00:00:00.019
▶ PRESS-02 2022-07-12 00:08:27.624
▶ ...
```

왜 쓰나 row[4]가 무슨 열인지 세어 보지 않아도 됩니다. row["전류"]라고 쓰면 코드가 스스로 설명하죠. 그리고 열 순서가 바뀌어도 안전합니다 — **이름으로 찾으니까요.**

응용 10일차에 배운 딕셔너리가 여기서 그대로 쓰입니다. row.get("유량", 0)처럼 **없는 열에 기본값**을 줄 수도 있죠. 그리고 DictReader는 헤더를 알아서 소비하므로 next()가 필요 없습니다.

```
import csv

with open("Data/08_press.csv", "r", encoding="utf-8") as f:
    reader = csv.DictReader(f)

    for row in reader:
        name = row["설비ID"]
        current = float(row["전류"])          # 이름으로 꺼내 형변환
        state = row.get("상태", "0")          # 없으면 기본값
        if state == "1":
            print(f"⚠ {name}: 전류 {current} · 상태 이상")
```

- ▶ ⚠ PRESS-10: 전류 21.4577 · 상태 이상
- ▶ ⚠ PRESS-11: 전류 -85.8307 · 상태 이상

주의

값은 여전히 **전부 문자열**입니다. DictReader를 써도 float() 변환은 필요합니다. 바뀌는 것은 "꺼내는 방법"이지 "자료형"이 아닙니다.

세 방법 비교

방법	헤더 처리	값 꺼내기	장단점
reader + next()	변수에 채기며 건너뛴	row[4]	헤더 재사용 가능 · 번호를 세야 함
reader + enumerate	버리고 건너뛴	row[4]	간단 · 헤더를 잃음
DictReader	자동 처리	row["전류"]	읽기 쉬움 · 열 순서에 안전

개념

입문 단계에서는 reader + next()로 원리를 익히고, 익숙해지면 DictReader로 넘어가는 순서가 좋습니다. 실무 코드는 열이 수십 개인 경우가 많아 번호로 세는 것이 현실적으로 불가능하거든요. 그리고 앞으로 배울 pandas는 아예 열 이름으로만 접근합니다 — df["전류"] 형태죠. DictReader가 그 감각의 예고편입니다.

STEP 6

오류와 예외의 이해

빨간 글씨는 안내문이다

교안 첫 장의 문장으로 시작합니다 — "오류는 잘못된 신호가 아니라, 컴퓨터가 보내는 안내문입니다." 숙련된 개발자도 매일 오류를 만납니다. 차이는 당황하지 않고 침착하게 대처하느냐에 있죠.

오류가 생기는 세 가지 경우

종류	언제 드러나나	예	난이도
문법 오류	실행되기 전에 발견	괄호 · 콜론 누락 · 들여쓰기 어긋남	쉬움
실행 중 오류	실행 도중 그 줄에서	0으로 나누기 · 없는 파일 열기	오늘의 핵심
결과 오류	드러나지 않음	문법 · 실행은 되지만 결과가 틀림	가장 까다로움

개념

세 번째가 지금까지 우리가 계속 만나온 것입니다 — 9일차의 `min_val = 0`, 10일차의 오경보, 12일차의 `[os.listdir()]`. 오류 메시지가 없어서 가장 잡기 어렵죠. 오늘 배우는 것은 두 번째, 실행 중 오류(예외)입니다. 이건 메시지가 나오므로 읽는 법만 알면 됩니다.

프로그램이 멈춘다는 것

예외가 발생하면 그 줄에서 즉시 중단되고, 아래 코드는 한 줄도 실행되지 않습니다. 교안이 굵게 강조한 부분입니다 — 30번째 줄에서 멈추면 31번째부터 100번째까지 전부 무시됩니다.

한 줄이 전체를 멈춘다

```
print("㉑ 시작")
```

```
value = int("스물")          # ← 여기서 ValueError, 즉시 중단
```

```
print("㉒ 이 줄은 실행되지 않는다")
```

```
print("㉓ 이 줄도 마찬가지")
```

▶ ㉑ 시작

▶ Traceback (most recent call last):

▶ ValueError: invalid literal for int() with base 10: '스물'

개념 트레이스백 읽는 순서 맨 아래에 진짜 원인이 있다

정의 빨간 글씨(Traceback)는 **아래에서 위로** 읽습니다. 맨 아랫줄에 오류 종류와 원인이, 그 위에 발생 위치가 있습니다.

```
Traceback (most recent call last):          # ① 안 읽어도 됨
  File "sensor.py", line 10, in <module>     # ② 그 다음 — 발생 위치
    temp = int(value)
ValueError: invalid literal for int() with base 10: '스물'
#   ↑ 오류 종류   ↑ 구체적인 설명   ← ③ 먼저 읽기
```

왜 쓰나 맨 윗줄 `Traceback (most recent call last):`는 **"지금부터 오류 추적을 보여준다"**는 안내일 뿐입니다. 읽어도 정보가 없죠. 진짜 정보는 맨 아랫줄에 있습니다.

응용 맨 아랫줄은 **콜론을 기준으로 두 부분**입니다. 앞은 **오류 종류**(`ValueError`), 뒤는 **구체적 설명**입니다. 종류를 알면 대응 방법이 정해지고, 설명을 보면 어떤 값이 문제였는지 알 수 있습니다.

```
# 오류 종류별로 읽어야 할 것

ValueError: invalid literal for int() with base 10: '스물'
#   → 종류: 값이 부적절   설명: '스물'을 int로 못 바꿈

FileNotFoundError: [Errno 2] No such file or directory: 'sensor.csv'
#   → 종류: 파일 없음   설명: sensor.csv 를 못 찾음

KeyError: '유량'
#   → 종류: 없는 키   설명: '유량' 키가 딕셔너리에 없음
```

개념

3일차에 배운 **"Traceback은 맨 아래가 진짜 원인"**이 그대로 재등장합니다. 그때는 오류를 없애는 법을 배웠다면, 오늘은 **오류를 다루는 법**을 배웁니다.

STEP 7

try-except 기본 구조

일어나면 그때 대응한다

교안이 짚은 **발상의 전환**이 핵심입니다 — "위험한 일이 안 일어나게 막는다"가 아니라, "일어나면 그때 대응한다." 어제까지는 `os.path.exists()`로 미리 막았다면, 오늘은 일어난 뒤에 받아내는 방법입니다.

키워드 `try - except` 시도해보고, 문제 생기면 이렇게

정의 실패할 수도 있는 코드를 `try` 블록에, 예외가 났을 때의 대처를 `except` 블록에 적습니다. 오류가 나도 프로그램은 멈추지 않습니다.

형태 `try:` 위험한 코드 `except:` 대처 방법

```
try:
    temp = int("스물")
except:
    print("못 바꿈")

print("프로그램은 계속 실행됩니다")
```

▶ 못 바꿈
▶ 프로그램은 계속 실행됩니다

왜 쓰나 `try` 안에서 오류가 나는 순간 즉시 `except`로 점프합니다. 그리고 `except` 블록을 실행한 뒤 그 아래 코드로 이어집니다. STEP 6에서 본 "한 줄이 전체를 멈춘다"가 해결되는 것이죠.

응용 `except` 안에서는 **대안 값을 넣어 두는 것**이 중요합니다. 오늘 수업 코드가 그 예입니다 — 변환에 실패하면 0으로 두고 계속 진행하죠. 아무 처리 없이 안내만 하면 다음 줄에서 또 터집니다.

```
temp = -1

try:
    temp = int("스물")
except:
    print("해봤는데 안되네요")
    temp = 0          # ← 대안 값을 넣어야 다음 줄이 안전하다

print(temp)
print(temp + 10)     # temp가 숫자라야 여기가 동작한다
```

```
▶ 해봤는데 안되네요
▶ 0
▶ 10
```

개념

try 블록에서 **오류가 나지 않으면** except는 건너뛵니다. 즉 **정상 상황에서는** try-except가 있으나 마나이고, 문제가 생겼을 때만 작동하는 **안전장치**입니다.

try-except 사용 원칙 세 가지

원칙	내용	이유
① 들여쓰기	콜론으로 끝내고 안쪽은 한 단계 들여쓰기	if·for와 동일한 규칙
② try는 작게	위험한 코드만 담는다	오류 위치 파악에 유리
③ 다 잡지 않기	모든 코드를 try로 감싸지 않는다	진짜 버그까지 숨겨진다

try는 작게

```
# x try가 너무 크다 — 어디서 터졌는지 알 수 없다
try:
    f = open(path, "r", encoding="utf-8")
    reader = csv.reader(f)
    header = next(reader)
    total = 0
    for row in reader:
        total += float(row[4])
    print(total / len(rows))
except:
    print("뭔가 잘못됐어요")      # ← 파일? 변환? 0으로 나누기? 알 수 없다

# ✓ 위험한 부분만 감싼다
for row in reader:
    try:
        total += float(row[4])    # 이 줄만 실패할 수 있다
    except ValueError:
        print(f"건너뛸: {row[0]}")
```

주의

③이 특히 중요합니다. **모든 코드를 try로 감싸면** 오타나 논리 버그까지 삼켜집니다. **프로그램은 안 멈추는데 결과는 계속 틀리게 되죠** — STEP 6의 "결과 오류"가 되어버립니다. 오늘 수업 코드의 첫 예제가 `except:` (종류를 안 적은 형태)인데, 이건 **원리를 보여주기 위한 것**이고 실전에서는 다음 STEP처럼 종류를 지정합니다.

STEP 8

구체적 예외 처리

무엇이 잘못됐는지 이름으로 안다

`except:` 뒤에 **예외 이름을 적으면 그 종류만** 잡습니다. 그러면 상황마다 정확한 안내를 줄 수 있고, 예상 못 한 다른 오류는 그대로 드러나 버그를 놓치지 않습니다.

예외 `ValueError` 값이 부적절할 때

정의 타입은 맞지만 **값의 내용이 부적절할 때** 납니다. 글자를 숫자로 바꿀 때 가장 자주 만납니다.

```
print(int("62"))          # 정상

int("스물")                # ValueError
int("3.5")                 # ValueError - int는 정수 문자열만
int("")                   # ValueError

▶ 62
▶ ValueError: invalid literal for int() with base 10: '스물'
```

왜 쓰나 교안 표현으로 "문자열이라는 종류는 맞지만, 내용이 숫자로 바꿀 수 없는 값이라 거부"되는 것입니다. 데이터 전처리에서 가장 많이 만나는 예외라, 교안이 "이 예외와 친해지는 것이 데이터 분석의 첫걸음"이라고 했습니다.

응용 CSV를 읽다 보면 빈 칸·N/A·-- 같은 값이 섞여 있습니다. 그 줄 하나 때문에 전체가 멈추지 않게 하려면 `ValueError`를 잡아 건너뛰면 됩니다.

```
rows = [ ["PRESS-01", "192.3"], ["PRESS-02", ""], ["PRESS-03", "N/A"],
["PRESS-04", "109.3"] ]
```

```
total = 0
count = 0
for name, value in rows:
    try:
        total += float(value)
        count += 1
    except ValueError:
        print(f" 건너뛴: {name} (값 = {value!r})")
```

```
print(f"유효 {count}건 · 합계 {round(total, 1)}")
```

- ▶ 건너뛴: PRESS-02 (값 = '')
- ▶ 건너뛴: PRESS-03 (값 = 'N/A')
- ▶ 유효 2건 · 합계 301.6

자주 하는 실수

`int("3.5")`도 `ValueError`입니다. `int`는 정수 형태 문자열만 받기 때문이죠. 소수점이 있는 값은 `float("3.5")`로 받거나 `int(float("3.5"))`로 두 번 거쳐야 합니다.

예외 `TypeError` 값의 종류 자체가 틀릴 때

정의 값의 종류(자료형) 자체가 맞지 않을 때 납니다. `ValueError`와 헷갈리기 쉽습니다.

```
10 + "개"          # TypeError - 숫자와 문자열을 더할 수 없다
int(None)          # TypeError - int는 None을 받지 않는다
```

```
int("스물")        # ValueError - 종류(str)는 맞고 내용이 문제
```

- ▶ `TypeError: unsupported operand type(s) for +: 'int' and 'str'`
- ▶ `ValueError: invalid literal for int() with base 10: '스물'`

왜 쓰나 구분 기준이 명확합니다 — 종류가 틀리면 `TypeError`, 종류는 맞고 내용이 부적절하면 `ValueError`입니다.

`int("스물")`은 문자열을 넘겼으니 종류는 맞고, 내용이 숫자가 아니라서 `ValueError`죠.

응용 `input()`은 항상 문자열을 돌려주므로, `int(input(...))`에서는 `TypeError`가 날 수 없습니다. 오늘 수업 코드에 `except TypeError`를 추가한 부분이 있는데, 그 자리에서는 도달할 수 없는 블록입니다 — STEP 10에서 다시 봅니다.

```
# input()은 항상 str → TypeError 불가
value = input("온도: ")      # 무엇을 입력해도 str
print(type(value).__name__)

# TypeError가 실제로 나는 상황
data = {"온도": None}
int(data["온도"])             # TypeError: NoneType

counts = [1, 2, "3"]
print(sum(counts))            # TypeError: int + str
```

개념

실무에서 `TypeError`는 `None`이 섞여 들어올 때 **가장 자주 만납니다**. **딕셔너리에서** `.get()`으로 꺼냈는데 키가 없어 `None`이 나오고, 그걸 계산에 쓰는 경우죠 — 10일차에 배운 기본값 이야기와 이어집니다.

예외 `FileNotFoundError` 파일이 없을 때

정의 `open`으로 열리는 파일이 **그 위치에 없을 때** 납니다.

```
open("sensor.csv", "r", encoding="utf-8")
# FileNotFoundError: [Errno 2] No such file or directory: 'sensor.csv'

# try-except로 받아내기
try:
    with open("sensor.csv", "r", encoding="utf-8") as f:
        data = f.read()
except FileNotFoundError:
    print("파일이 없습니다. 경로를 확인하세요.")
    data = ""
```

▶ **파일이 없습니다. 경로를 확인하세요.**

왜 쓰나 교안이 짙은 대로 **"설비 데이터는 거의 항상 CSV 파일을 열며 시작"**하므로, 첫 단계에서 이 예외에 대비하는 것이 중요합니다.

응용 원인이 대개 세 가지입니다 — ① **파일명 오타** ② **경로 문제** ③ **확장자 숨김**. 특히 ③은 윈도우에서 `.csv`가 안 보여 `sensor`인 줄 알았는데 실제로는 `sensor.csv`인 경우입니다. 어제 배운 `os.getcwd()`·`os.listdir()`이 진단 도구가 됩니다.

```
import os

path = "Data/sensor.csv"
try:
    with open(path, "r", encoding="utf-8") as f:
        data = f.read()
except FileNotFoundError:
    print(f"[안내] {path} 를 찾을 수 없습니다.")
    print(" 현재 위치:", os.getcwd())
    print(" Data 폴더 목록:", os.listdir("Data"))
```

- ▶ [안내] Data/sensor.csv 를 찾을 수 없습니다.
- ▶ 현재 위치: /home/me/project
- ▶ Data 폴더 목록: ['08_press.csv', 'sample.txt']

주의

어제 배운 `os.path.exists()`로 미리 막는 것과 `try-except`로 일어난 뒤 받는 것, 둘 다 유효합니다. 둘의 차이는 STEP 9 뒤에서 정리합니다.

자주 만나는 예외 여섯

예외	발생 상황	예시	오늘 만난 곳
<code>ValueError</code>	값이 부적절	<code>int("스물")</code>	CSV 형변환
<code>FileNotFoundError</code>	파일이 없음	<code>open</code> 실패	CSV 열기
<code>ZeroDivisionError</code>	0으로 나눔	<code>10 / 0</code>	데이터 0개일 때 평균
<code>IndexError</code>	범위 초과	<code>row[5]</code>	칸이 부족한 줄
<code>KeyError</code>	없는 키 접근	<code>row["유량"]</code>	<code>DictReader</code>
<code>TypeError</code>	값 종류가 틀림	<code>10 + "개"</code>	<code>None</code> 이 섞일 때

개념

교안이 "지금 다 외울 필요 없다"고 못 박았습니다 — "프로그래밍은 암기 과목이 아니다. 빨간 글씨를 만나면 맨 아랫줄을 읽어 종류를 확인하면 된다. 두세 번 만나다 보면 자연스럽게 외워진다." 외우려 하지 말고 만날 때마다 정리하는 습관을 만드는 편이 낫습니다. 이 표는 그 정리의 시작점입니다.

STEP 9

여러 예외 동시 처리

순서 · 구분 · 묶기

실무 데이터는 **한 가지 오류만 나지 않습니다**. 어떤 줄은 글자가 섞여 `ValueError`, 어떤 줄은 칸이 부족해 `IndexError`가 납니다. 각각 다른 안내를 주고 싶으면 `except`를 여러 개 나란히 적습니다.

키워드 `except` 여러 개 예외마다 다른 대처

정의 `except`를 여러 개 쓰면 파이썬이 위에서부터 확인해 일치하는 첫 블록을 실행합니다.

형태 `try: ... except ValueError: ... except IndexError: ...`

```
rows = [{"PRESS-01", "192.3"}, {"PRESS-02"}, {"PRESS-03", "N/A"}]

for row in rows:
    try:
        name = row[0]
        value = float(row[1])
        print(f"{name}: {value}")
    except ValueError:
        print(f"{row[0]}: 숫자가 아닙니다")
    except IndexError:
        print(f"{row[0]}: 칸이 부족합니다")
```

- ▶ PRESS-01: 192.3
- ▶ PRESS-02: 칸이 부족합니다
- ▶ PRESS-03: 숫자가 아닙니다

왜 쓰나 같은 반복문 안에서 **줄마다 다른 문제**가 나도 각각 맞는 안내가 나갑니다. 그리고 어느 쪽이든 **다음 줄로 계속** 진행합니다.

응용 같은 방식으로 처리할 예외들은 **괄호로 묶어** 한 블록에서 받을 수 있습니다. 안내 문구가 같다면 굳이 나눌 필요가 없죠.

```
for row in rows:
    try:
        value = float(row[1])
        print(f"{row[0]}: {value}")
    except (ValueError, IndexError):      # 괄호로 묶기
        print(f"{row[0]}: 데이터 이상 — 건너뛰니다")
```

-
- ▶ PRESS-01: 192.3
 - ▶ PRESS-02: 데이터 이상 — 건너뛰니다
 - ▶ PRESS-03: 데이터 이상 — 건너뛰니다

자주 하는 실수

순서가 중요합니다. 파이썬은 위에서부터 확인하므로, **넓은 예외를 위에 두면 아래 구체적 예외는 영원히 실행되지 않습니다.**

순서 원칙 — 구체적인 것이 위로

순서에 따른 차이

```
# x 넓은 예외가 위 — 아래는 도달 불가
try:
    int("스물")
except Exception:      # 모든 예외를 여기서 다 잡는다
    print("무언가 잘못됐습니다")
except ValueError:
    print("이 줄은 절대 실행되지 않는다")
```

```
# ✓ 구체적인 것이 위
try:
    int("스물")
except ValueError:      # 정확한 안내가 가능
    print("숫자를 입력해 주세요")
except Exception:      # 그 밖의 예상 못 한 예외
    print("예상치 못한 오류")
```

-
- ▶ 무언가 잘못됐습니다
 - ▶ 숫자를 입력해 주세요

원칙	내용	왜
① 순서	구체적 예외를 위에, 넓은 예외를 아래에	위에서부터 확인하므로
② 구분	예외마다 다른 안내를 준다	뭉뚱그리면 예상 못한 버그를 놓침
③ 묶기	같은 방식으로 처리할 것은 괄호로	<code>except (ValueError, IndexError):</code>

예방과 대응 — 어느 쪽을 쓸 것인가

어제 배운 `os.path.exists()`로 미리 막는 방법과 오늘 배운 `try-except`로 일어난 뒤 받는 방법이 있습니다. 둘은 경쟁 관계가 아니라 쓰이는 자리가 다릅니다.

	예방 (exists 등)	대응 (try-except)
시점	실행 전에 확인	실패한 뒤에 받아냄
잘 맞는 곳	파일 존재 · 키 존재처럼 미리 알 수 있는 것	값 변환 · 계산처럼 해 봐야 아는 것
한계	확인하고 실행 사이에 상황이 바뀔 수 있음	남용하면 진짜 버그를 숨김
오늘 예	<code>if os.path.exists(path):</code>	<code>except FileNotFoundError:</code>

설비 적용

실무에서는 둘을 겹쳐 씁니다. 파일은 `exists`로 먼저 확인해 명확한 안내를 주고, 그래도 열다가 실패할 수 있으니 `try-except`로 한 번 더 받습니다. 그리고 반복문 안의 값 변환은 반드시 `try-except`입니다 — 수만 줄을 하나하나 미리 검사하는 것보다 실패한 줄만 건너뛰는 편이 훨씬 간단하니까요. CHAPTER 0에서 본 "50줄 중 10번째에서 멈춤" 문제의 답이 바로 이것입니다.

예방 + 대응을 겹쳐 쓰기

```
import csv, os
```

```
path = os.path.join("Data", "08_press.csv")
```

```
if not os.path.exists(path):
```

 # ① 예방

```
    print("파일이 없습니다:", path)
```

```
else:
```

```
    total, ok, skipped = 0.0, 0, 0
```

```
    with open(path, "r", encoding="utf-8") as f:
```

```
        reader = csv.reader(f)
```

```
        next(reader)
```

```
        for row in reader:
```

```
            try:
```

 # ② 대응

```
                total += float(row[4])
```

```
                ok += 1
```

```
            except (ValueError, IndexError):
```

```
                skipped += 1
```

```
    print(f"처리 {ok}건 · 건너뛴 {skipped}건 · 전류 합계 {round(total, 2)}")
```

▶ 처리 12건 · 건너뛴 0건 · 전류 합계 -63.86

STEP 10

내 코드 점검 — 수업 코드 2개

파일명 오타와 bare except

오늘 만든 수업 파일은 `19_file_io.py`와 `20_handler.py`입니다. 내용은 정확하고 주석도 훌륭합니다 — 특히 `next(reader)`와 `DictReader` 설명은 교안보다 더 자세할 정도입니다. 짚을 곳은 세 군데입니다.

① 파일명에 마침표 대신 쉼표가 들어갔습니다

	현재	올바른 이름
파일명	<code>19_file_io.py</code>	<code>19_file_io.py</code>
확장자	없음 — 쉼표는 확장자 구분자가 아니다	<code>.py</code>
결과	편집기가 파이썬으로 인식 못 함 · import 불가	정상

주의

`python "19_file_io.py"`처럼 따옴표로 감싸면 실행은 됩니다. 파이썬은 확장자를 강제하지 않으니까요. 하지만 VS Code가 파이썬 파일로 인식하지 못해 문법 강조·자동완성·오류 표시가 전부 꺼집니다. 어제 `18_module_1 copy.py`의 공백을 고치신 것과 같은 종류의 문제라, 이것도 이름만 바꾸면 끝납니다.

② bare except — 종류를 안 적은 except

첫 try 예제가 `except:` 형태입니다. 교안 09_01 PART 2의 코드를 그대로 따라 하신 것이라 틀린 것은 아닙니다. 다만 이 형태의 위험을 알아 두면 좋습니다.

제출 코드 — 동작은 정상

```
temp = -1

try:
    temp = int("스물")
except:
    # ← 종류를 안 적었다
    print("해봤는데 안되네요")
    temp = 0

print(temp)
```

▶ 해봤는데 안되네요
▶ 0

bare except 가 숨기는 것

```
# 위험한 이유 — 오타(NameError)까지 삼킨다
try:
    value = undefined_name + 1    # 변수 이름 오타 = 진짜 버그
except:
    print("해봤는데 안되네요")    # ← 버그가 조용히 묻힌다

# ✓ 종류를 지정하면 진짜 버그는 드러난다
try:
    value = undefined_name + 1
except ValueError:
    print("숫자 변환 실패")

# NameError: name 'undefined_name' is not defined ← 제대로 터진다
```

▶ 해봤는데 안되네요
▶
▶ NameError: name 'undefined_name' is not defined

개념

STEP 7의 원칙 ③ "다 잡지 않기"가 이것입니다. `except:`는 모든 예외를 다 잡으므로 오타·논리 버그까지 "처리됨"으로 만들어 버립니다. 프로그램은 안 멈추는데 결과는 계속 틀리게 되죠. 교안 09_02에 이 주제(bare `except`의 위험)를 다루는 장이 따로 있는데, **내일 배울 내용**입니다. 오늘 코드는 그대로 두시고, 앞으로 새로 쓸 때는 `except ValueError:`처럼 종류를 적는 습관을 들이면 됩니다.

③ `except TypeError` 는 도달할 수 없는 블록입니다

실습 2의 두 번째 버전에 `except TypeError`를 추가하셨는데, 그 자리에서는 절대 실행되지 않습니다.

제출 코드 — 실습 2 두 번째 버전

```
origin = input("온도 : ")      # input()은 항상 str을 돌려준다

try:
    temp = int(origin)
except ValueError:
    print("숫자 아니면 왜 저를 부르셨어요? 0으로 생각할게요")
except TypeError:
    # ← 도달 불가
    print("타입 문제는 전지구적 문제입니다.")
```

왜 도달 불가인가

```
# input()이 무엇을 돌려주는지 확인
value = input("온도: ")
print(type(value).__name__)    # 무엇을 입력해도 str

# int(str) 에서 나는 예외는 ValueError 뿐
int("스물")      # ValueError
int("3.5")       # ValueError
int("")          # ValueError

# TypeError는 str이 아닌 것을 넘겨야 난다
int(None)        # TypeError
int([1, 2])      # TypeError
```

- ▶ str
- ▶ ValueError: invalid literal for int() with base 10: '스물'
- ▶ TypeError: int() argument must be a string ... not 'NoneType'

응용

시도 자체는 좋습니다 — 교안 PART 3이 "여러 except로 나눠 처리"를 다뤘으니 정확히 그 연습이죠. 다만 이 자리에서 실제로 함께 날 수 있는 예외는 TypeError가 아니라 ZeroDivisionError입니다. **입력받은 값으로 나눗셈을 하면 0을 넣었을 때 나거든요.** 선택문제인 09_01 실습 3이 정확히 그 조합을 요구합니다 — STEP 11에 완성본을 실었습니다.

STEP 11

실습 점검 — 기본 5 · 선택 2

원본 CSV 오염 사고 포함

오늘 실습과제는 **기본문제 5개(08_02 실습 1~5) 중 3개 이상**, 선택문제 2개(08_02 실습 6, 09_01 실습 3)입니다. 08_02_example.py에 여섯 문제를 모두 담으셨으니 **요건은 충분히 넘겼습니다**. 다만 한 건은 데이터를 실제로 망가뜨렸습니다.

구분	문제	제출	상태
기본	실습 1 · open으로 파일 읽기	○	readlines 인자 오해 · close 누락
기본	실습 2 · with open으로 쓰기	○	줄바꿈 누락
기본	실습 3 · a 모드로 이어쓰기	○	정확
기본	실습 4 · csv.reader로 읽기	○	정확
기본	실습 5 · csv.writer로 쓰기	○	원본 CSV 오염
선택	실습 6 · CSV 읽어 조건 저장	○	저장 · 필터 누락
선택	09_01 실습 3 · 구체적 예외 검증	—	미제출

실습 5 — 원본 센서 데이터가 오염됐습니다

가장 시급한 문제입니다. 결과를 새 파일이 아니라 원본 08_press.csv에 a 모드로 쓰셨습니다. 그래서 실행할 때마다 두 줄씩 쌓였습니다.

제출 코드 — 실습 5

```
with open("Data/08_press.csv", "a", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(["시각", "설비"])
    writer.writerow(["09:00", "PUMP-01"])
```

현재 Data/08_press.csv 상태 — 39줄

```
설비ID, 시각, 진동X, 진동Y, 전류, 상태
PRESS-01, 2022-07-12 00:00:00.019, 0.117, -0.1764, 192.3387, 0
...
PRESS-12, 2022-07-17 11:04:12.331, 0.1653, -0.2211, -53.9012, 1
시각, 설비      ← 여기부터 오염
09:00, PUMP-01
시각, 설비
09:00, PUMP-01
시각, 설비      (5번 실행 → 10줄 추가)
09:00, PUMP-01
```

자주 하는 실수

문제가 세 겹입니다. ① **원본 데이터에 썼습니다** — 결과는 반드시 새 파일로 가야 합니다. ② **a 모드라 누적됩니다** — **요구사항은 w였습니다**. ③ **열 개수가 안 맞습니다** — 원본은 6열인데 2열짜리 행이 들어가 표 구조가 깨졌습니다. 실행할 때마다 조용히 커지고 **오류는 한 번도 나지 않습니다**. 지금은 실습 데이터라 다시 받으면 되지만, 현장 데이터였다면 복구가 어려운 사고입니다.

고친 코드

```
import csv

# ✓ 결과는 새 파일에 · w 모드 · newline 필수
with open("Data/result.csv", "w", encoding="utf-8", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["시각", "설비"])
    writer.writerow(["09:00", "PUMP-01"])

# 확인
with open("Data/result.csv", "r", encoding="utf-8") as f:
    print(f.read())
```

```
▶ 시각, 설비
▶ 09:00, PUMP-01
```

주의

복구 방법: 오염된 줄은 전부 파일 끝에 몰려 있으므로, 08_press.csv를 열어 시각, 설비 이후를 지우면 됩니다. 원본은 12행 + 헤더 1행 = **13줄**이 맞습니다(교안 코드의 len(lines) 출력이 13). 앞으로는 **원본 데이터 파일은 r** 모드로만 열기를 **규칙으로 삼는 게 안전합니다**.

실습 1 — readlines(5)는 "5줄"이 아닙니다

제출 코드 — 실습 1

```
f = open("Data/08_press.csv", "r", encoding="utf-8")
print(f.readlines(5))          # 5줄을 읽으려는 의도
# f.close() 가 없다
```

- ▶ ['설비ID, 시각, 진동X, 진동Y, 전류, 상태\n']
- ▶
- ▶ ↑ 1줄만 나온다

호출	뜻	결과
<code>readlines()</code>	전체를 줄 단위로	13줄 전부
<code>readlines(5)</code>	약 5바이트까지만 읽는다 (hint)	1줄
<code>readlines(100)</code>	약 100바이트까지	3줄 — 줄 수와 무관
<code>readlines()[:5]</code>	전체를 읽고 앞 5줄만	5줄

개념

괄호 안 숫자가 "줄 수"가 아니라 "바이트 수 힌트"라는 것이 함정입니다. 파이썬은 그 바이트를 채울 때까지 줄을 읽고, 줄 중간에서 자르지 않으려고 마지막 줄은 끝까지 읽습니다. 그래서 5를 주면 첫 줄 하나만 나오죠. **N줄만 보고 싶으면 슬라이싱(`readlines()[:5]`)을 쓰는 것이 정확합니다.** 그리고 `f.close()`가 빠져 있는데, `with` 버전을 주석으로 남겨 두셨으니 그쪽을 살리면 한 번에 해결됩니다.

실습 2 — 줄바꿈이 빠져 붙어 나옵니다

제출 코드 — 실습 2

```
with open("Data/sample.txt", "w", encoding="utf-8") as f:
    f.write("\n".join(["123", "456"]))    # 123\n456
    f.write("\n1\n")                     # 123\n456\n1\n
    f.writelines(["1", "2", "3"])         # ← \n 이 없어 123 이 한 줄로
```

- ▶ ['123\n', '456\n', '1\n', '123']
- ▶
- ▶ ↑ 마지막이 '1', '2', '3' 이 아니라 '123' 한 줄

개념

writelines()는 이름에 lines가 들어가 있어 **줄바꿈을 넣어줄 것 같지만 넣지 않습니다**. 리스트의 각 문자열을 그냥 이어 붙일 뿐이죠. 줄을 나누려면 각 요소에 \n이 있거나 "\n".join()을 써야 합니다. 요구사항 ④가 "r" 모드로 다시 열어 쓴 내용을 확인"이었는데, **확인해 보셨기 때문에 이 현상이 파일에 그대로 남아 있는 것**이기도 합니다 — 확인 단계를 거친 건 좋은 습관입니다.

실습 6 — 저장이 빠졌습니다

요구사항은 네 단계였습니다 — ① csv import ② reader로 읽고 헤더 건너뛰기 ③ **float 변환 후 기준(90) 초과 행만 리스트에 모으기** ④ **writer로 새 CSV에 저장**. 제출 코드는 ②까지만 하고 ③④가 빠졌습니다.

제출 코드 — 실습 6

```
with open(csv_path, "r", encoding="utf-8") as f:
    reader = csv.reader(f)
    for idx, row in enumerate(reader):
        if idx == 0:
            continue                # ② 헤더 건너뛰기 — OK
        data_range = row[1:-1]
        if len(data_range) > 0 and data_range[0] != "":
            print(data_range)        # ③ float 변환·기준 90 없음
                                    # ④ 저장 없음
```

완성본 — 오전과 오후를 합친 코드

```
import csv, os

src = os.path.join("Data", "08_press.csv")
dst = os.path.join("Data", "over_90.csv")

rows = []
with open(src, "r", encoding="utf-8") as f:
    reader = csv.reader(f)
    header = next(reader)          # ② 헤더를 챙기며 건너뛰다

    for row in reader:
        try:
            current = float(row[4])    # ③ float 변환
        except (ValueError, IndexError): # 오후에 배운 안전장치
            continue
        if abs(current) > 90:          # 기준 90 초과
            rows.append(row)

with open(dst, "w", encoding="utf-8", newline="") as f:    # ④ 새 CSV로 저장
    writer = csv.writer(f)
```

```
writer.writerow(header)
writer.writerows(rows)
```

```
print(f"원본 {len(rows)}건 저장 → {dst}")
```

▶ 원본 7건 저장 → Data/over_90.csv

▶

▶ (over_90.csv)

▶ 설비ID, 시각, 진동X, 진동Y, 전류, 상태

▶ PRESS-01,2022-07-12 00:00:00.019,0.117,-0.1764,192.3387,0

▶ ...

연결

이 코드가 **오늘 하루의 결론**입니다. `next()`로 헤더를 챙기고(오전), `float()`로 변환하고(오전), `try-except`로 이상한 줄을 건너뛰고(오후), `writer`로 새 파일에 저장합니다(오전). **찾기 → 열기 → 읽기 → 처리 → 저장**
사이클이 한 화면에 들어 있습니다.

09_01 실습 3 — 미제출 (선택문제)

요구사항은 `ValueError`와 `ZeroDivisionError`를 각각 다른 `except`로 구분해 처리하는 것입니다. **오늘 만든 실습 2 코드에서** `except TypeError`를 `except ZeroDivisionError`로 바꾸고 나눗셈을 넣으면 됩니다.

실습 3 완성본 — 세 가지 입력 모두 검증

```
origin = input("온도 : ")
temp = 0

try:
    temp = int(origin)                # ① int 변환을 try에
    ratio = 100 / temp                # 0을 넣으면 여기서 터진다
    print(f" 변환 성공 → {temp}, 100/{temp} = {round(ratio, 2)}")
except ValueError:                   # ② 숫자가 아닐 때
    print(" 숫자를 입력해 주세요. 0으로 처리합니다.")
except ZeroDivisionError:            # ③ 0을 넣었을 때
    print(" 0으로는 나눌 수 없습니다. 계산을 건너뛰니다.")

print(f" → 프로그램은 계속 진행됩니다 (temp={temp})") # ④ 멈추지 않음
```

-
- ▶ [입력 25] 변환 성공 → 25, 100/25 = 4.0
 - ▶ → 프로그램은 계속 진행됩니다 (temp=25)
 - ▶
 - ▶ [입력 스물] 숫자를 입력해 주세요. 0으로 처리합니다.
 - ▶ → 프로그램은 계속 진행됩니다 (temp=0)
 - ▶
 - ▶ [입력 0] 0으로는 나눌 수 없습니다. 계산을 건너뛰니다.
 - ▶ → 프로그램은 계속 진행됩니다 (temp=0)

치트시트

파일 입출력과 예외처리 한 장 요약

파일 읽기·쓰기

	문법	돌려주는 것 / 하는 일
메서드	<code>f.read()</code>	전체를 한 문자열로
메서드	<code>f.readline()</code>	한 줄을 문자열로 (부를 때마다 다음 줄)
메서드	<code>f.readlines()</code>	전체를 줄 단위 리스트로 · 인자는 바이트 hint
메서드	<code>f.write("문자열")</code>	문자열 하나 쓰기 · 줄바꿈 자동 없음
메서드	<code>f.writelines([...])</code>	여러 문자열 쓰기 · 줄바꿈 자동 없음
문법	<code>for line in f:</code>	한 줄씩 순회 — 큰 파일에 유리

CSV

	문법	설명
내장함수	<code>import csv</code>	표준 라이브러리 — 설치 불필요
읽기	<code>reader = csv.reader(f)</code>	각 행을 리스트로 · 값은 전부 문자열
읽기	<code>header = next(reader)</code>	첫 줄을 챙기며 건너뛰다
읽기	<code>csv.DictReader(f)</code>	행을 딕셔너리로 · 열 이름으로 접근
쓰기	<code>writer = csv.writer(f)</code>	<code>open</code> 에 <code>newline=""</code> 필수
쓰기	<code>writer.writerow([...])</code>	리스트를 한 행으로
쓰기	<code>writer.writerows([[...]])</code>	여러 행을 한 번에

예외처리

	문법	설명
키워드	<code>try:</code>	실패할 수 있는 코드 — 작게 유지
키워드	<code>except 예외이름:</code>	그 종류만 잡는다 — 권장

	문법	설명
키워드	<code>except (A, B):</code>	같은 방식으로 처리할 것을 묶기
키워드	<code>except:</code>	모든 예외 — 진짜 버그까지 숨김
원칙	구체적 → 넓은 순서	위에서부터 확인하므로 순서가 중요

자주 만나는 예외 여섯

예외	상황	대표 원인
<code>ValueError</code>	값이 부적절	CSV 문자열을 float로 변환 실패
<code>FileNotFoundError</code>	파일이 없음	파일명 오타 · 경로 · 확장자 숨김
<code>ZeroDivisionError</code>	0으로 나눔	데이터 0개인데 평균 계산
<code>IndexError</code>	범위 초과	칸이 부족한 줄에서 <code>row[4]</code>
<code>KeyError</code>	없는 키 접근	<code>DictReader</code> 에서 없는 열 이름
<code>TypeError</code>	값 종류가 틀림	<code>None</code> 이 계산에 섞여 들어옴

오늘의 완성형 — 읽어서 걸러 저장하기

외워둘 한 덩어리

```
import csv, os

src = os.path.join("Data", "08_press.csv")
dst = os.path.join("Data", "over_90.csv")

if not os.path.exists(src):                # ① 예방
    print("파일 없음:", src)
else:
    rows = []
    with open(src, "r", encoding="utf-8") as f:    # ② 열기
        reader = csv.reader(f)
        header = next(reader)                    # ③ 헤더 챙기기
        for row in reader:
            try:                                    # ④ 대응
                if abs(float(row[4])) > 90:        #      처리
                    rows.append(row)
            except (ValueError, IndexError):
                continue

    with open(dst, "w", encoding="utf-8", newline="") as f:    # ⑤ 저장
        writer = csv.writer(f)
        writer.writerow(header)
        writer.writerows(rows)

print(f"{len(rows)}건 저장 → {dst}")
```

오류·함정 사전

한 번씩은 반드시 겪는 것들

B-1 · 오늘 실제로 겪은 것

x 이렇게 하면	무슨 일이	✓ 바로잡기
원본 CSV를 a 모드로 열기	실행할 때마다 원본이 오염 — 오류 없음	결과는 새 파일에 w 모드로
f.readlines(5)	5줄이 아니라 1줄 — 인자는 바이트 hint	f.readlines()[:5]
f.writelines(["1", "2", "3"])	123 이 한 줄로 붙는다	"\n".join([...]) 또는 각 요소에 \n
f = open(...) 후 close 없음	자원 누수 · 쓰기 내용 유실 가능	with open(...) as f:
except TypeError	int(str)에서는 도달 불가능한 블록	except ZeroDivisionError
except:	오타·논리 버그까지 삼킨다	예외 종류를 지정
파일명에 심표 (...io.py)	편집기가 파이썬으로 인식 못 함	마침표로 수정

B-2 · CSV 혼한 오류 네 가지 (교안)

증상	원인	해결
한글 깨짐	인코딩 불일치	utf-8 ↔ cp949 바꿔 보기
빈 줄 끼임	newline 누락 (윈도우)	쓰기 open 에 newline=""
계산 오류	값이 문자열 상태	float() · int() 변환
첫 줄 오류	헤더까지 데이터로 처리	next(reader) 또는 DictReader

B-3 · 파일 함정

x 이렇게 하면	무슨 일이	✓ 바로잡기
open(path) # 없는 파일	FileNotFoundError로 즉시 중단	exists 확인 또는 try-except

✗ 이렇게 하면	무슨 일이	✓ 바로잡기
"w" 로 로그 열기	실행할 때마다 앞 기록이 사라짐	"a" 모드
f.write(123)	TypeError — write는 문자열만	f.write(str(123))
열 개수가 다른 행 쓰기	표 구조가 어긋남 — 오류 없음	모든 행의 열 수를 맞춘다
float("192.3") > "100"	문자열 비교라 결과가 뒤집힐 수 있음	양쪽 다 숫자로 변환

개념

오늘 부록 B의 절반이 **오류 없이 결과만 틀리는 것들**입니다 — CSV 오염, readlines(5), writelines 줄바꿈, 문자열 비교, 열 개수 불일치. 예외처리를 배웠다고 이런 것까지 잡히지는 않습니다. try-except는 **터지는 오류**를 다루는 도구이고, **조용히 틀리는 것**은 여전히 예상 출력과 대조하는 수밖에 없습니다.

실습 하이라이트 & 다음 시간 예고

사이클 완성, 그리고 견고하게

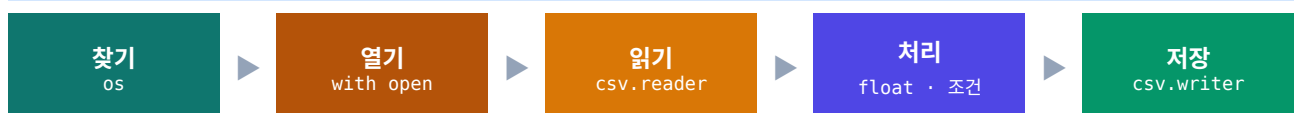
C-1 · 오늘 자료와 작성 파일

파일	내용	상태
Lecture/08_02_파일_입출력	read 3종 · csv.reader/writer · DictReader (32p 나머지)	전 범위 완료
Lecture/09_01_예외처리_기초	오류 3종 · 트레이스백 · try-except · 구체적인 예외 (23p)	전 범위 완료
Lecture/09_02_견고한_예외처리	else · finally · bare except · as e · raise (27p)	진도 밖 — 내일
Python/19_file_io.py	read/write · csv.reader/writer/DictReader · next()	정확 · 파일명 오타
Python/20_handler.py	트레이스백 3종 · try-except · 실습 2	정확 · bare except
Data/08_press.csv	소성가공 프레스 실데이터	실습 5로 오염 — 복구 필요
Practice/08_02_example.py	실습 1~6 전부	6/6 제출 · 버그 4건

C-2 · 오늘 배운 것 한눈에

분류	오늘 나온 것	개수
파일 메서드	read · readline · readlines · write · writelines	5개
csv	reader · writer · writerow · writerows · DictReader	5개
내장 함수	next() · float() · abs()	3개
키워드	try · except	2개
예외 종류	ValueError · TypeError · FileNotFoundError 외 3	6개
개념	오류 3종류 · 트레이스백 읽는 순서 · 예방 vs 대응	3개

C-3 · 데이터 처리 한 사이클 — 완성



12일차에 앞의 두 칸을, 오늘 뒤의 세 칸을 채웠습니다. 그리고 try-except가 이 사이클 전체를 감싸는 안전장치로 붙었습니다. 앞으로 배울 pandas는 이 다섯 단계를 각각 한 줄로 줄여줄 뿐, 구조는 똑같습니다.

단계	지금 (순수 파이썬)	앞으로 (pandas)
읽기	<code>csv.reader + next()</code>	<code>pd.read_csv(path)</code>
처리	<code>for + float + if</code>	<code>df[df["전류"] > 90]</code>
저장	<code>csv.writer + writerows</code>	<code>df.to_csv(path)</code>

C-4 · 다음 시간 예고 — 09_02 견고한 예외처리

다음 주제	한 줄 미리보기
else 와 finally	try가 성공했을 때만 도는 else, 성공·실패와 무관하게 항상 도는 finally
bare except의 위험	오늘 수업 코드에 쓴 except: 형태를 왜 피해야 하는지 — STEP 10에서 미리 봤다
as e 로 에러 정보 꺼내기	except ValueError as e: — 무엇이 왜 실패했는지 메시지를 받아 로그에 남긴다
raise 로 직접 발생시키기	기준을 어긴 값을 만나면 내가 예외를 던진다 — 검증 로직의 기본
반복문 안 예외처리	CHAPTER 0의 "50줄 중 10번째" 문제를 정면으로 다루는 장

연결

09_02 교안 첫 장에 "이 단원의 실데이터 — ICT 검사"가 나옵니다. 데이터가 하나 더 늘어난다는 뜻이죠. 그리고 오늘 STEP 10에서 짚은 bare except와 STEP 9에서 다룬 except 순서가 그 교안의 핵심 주제입니다 — 오늘 코드에 남은 것을 내일 정리하게 됩니다. 미리 알고 가면 훨씬 잘 들어옵니다.

오늘의 성취와 다음 한 걸음

파일을 다루면 오류를 만난다

오늘 배운 것을 한 문장으로 줄이면 — **파일에서 표를 읽어 숫자로 바꾸고, 조건에 맞는 것만 새 파일로 저장하되, 중간에 이상한 값이 나와도 멈추지 않는다.** 12일차에 시작한 데이터 처리 사이클이 오늘 완성됐습니다.

그리고 오후에 배운 예외처리가 **왜 하필 파일 다음인지**도 분명해졌습니다. 파일은 없을 수도, 깨져 있을 수도, 이상한 값이 섞여 있을 수도 있습니다. 강사님이 어제 예고하신 그대로 **"파일을 다루다 보면 에러가 먼저 발생"**하니까요.

오늘 헛갈렸던 지점 다시 짚기

헛갈린 것	정리
<code>read · readline · readlines</code>	끝의 s 한 글자가 결과를 바꾼다 — 문자열 / 문자열 / 리스트
<code>readlines(5)</code> 의 5	줄 수가 아니라 바이트 힌트 . N줄만 보려면 <code>readlines()[:N]</code>
<code>w</code> 와 <code>a</code>	<code>w</code> 는 지우고 새로 씀, <code>a</code> 는 이어 붙임 . 원본 데이터는 <code>r</code> 로만 여는 것이 안전
왜 <code>newline=""</code>	윈도우에서 줄바꿈이 이중 변환돼 빈 줄이 낀다 . 쓰기에는 항상 붙인다
csv 값이 문자열	<code>reader</code> 든 <code>DictReader</code> 든 전부 문자열 . 계산 전에 <code>float()</code> 필수
<code>ValueError</code> vs <code>TypeError</code>	종류가 틀리면 <code>TypeError</code> , 종류는 맞고 내용이 부적절하면 <code>ValueError</code>
<code>except</code> 순서	구체적인 것이 위로. 넓은 예외를 위에 두면 아래는 도달 불가
예방 vs 대응	파일 존재는 <code>exists</code> 로 예방 , 값 변환은 <code>try-except</code> 로 대응 . 겹쳐 쓴다

개념

복습 루틴 추천: ① `Data/08_press.csv` 복구 — 시각, 설비 이후 10줄을 지워 13줄로 되돌리기 (가장 먼저) → ② `readlines() · readlines(5) · readlines()[ :5]`를 나란히 출력해 차이 확인 → ③ `csv.reader`로 읽은 값에 `float()` 없이 `> 90`을 비교해 **틀린 결과가 나오는 것을** 눈으로 보기 → ④ 미제출인 **09_01 실습 3**을 STEP 11을 덮고 직접 풀기 → ⑤ 부록 A의 완성형 코드를 처음부터 쳐 보며 다섯 단계가 어디에 있는지 짚기. ①이 오늘 가장 급한 일입니다.